

Euclidean (L2) Distance, Worked Out by Hand

Straight-line distance in embedding space — the metric that measures how far apart two vectors sit rather than how they are angled — computed by hand, with its exact algebraic link to cosine once vectors are normalised.

What this document does. It computes the L2 distance from a query to three documents, contrasts the ranking it produces with the cosine ranking on the same vectors (they disagree — instructively), and then derives the identity $\|\hat{\mathbf{u}} - \hat{\mathbf{v}}\|^2 = 2 - 2\cos$ that makes Euclidean and cosine *equivalent rankers* on unit-length embeddings. Numbers from Appendix A; vectors match the cosine, dot-product, and normalisation PDFs.

1 The formula

The Euclidean (or L_2) distance between $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ is the length of the difference vector:

$$d_2(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\| = \sqrt{\sum_{i=1}^n (u_i - v_i)^2}$$

It is the ordinary ruler distance generalised to n dimensions. Small distance = close = similar; it ranges in $[0, \infty)$ and is 0 only for identical vectors.

Intuition

Cosine asks “same *direction*?”; Euclidean asks “same *location*?” Two vectors can point the same way yet sit far apart if one is much longer — so the two metrics need not agree. Euclidean cares about the gap in every coordinate, magnitude included; a vector that is “the right meaning but much bigger” is, to Euclidean, genuinely far away.

2 Worked by hand

Query $\mathbf{q} = (2, 1, 2)$ against $\mathbf{a} = (1, 0, 2)$, $\mathbf{b} = (0, 3, 1)$, and $\mathbf{c} = (4, 2, 4) = 2\mathbf{q}$.

$$\mathbf{q} - \mathbf{a} = (1, 1, 0) \Rightarrow d = \sqrt{1 + 1 + 0} = \sqrt{2} = 1.4142,$$

$$\mathbf{q} - \mathbf{b} = (2, -2, 1) \Rightarrow d = \sqrt{4 + 4 + 1} = \sqrt{9} = 3.0000,$$

$$\mathbf{q} - \mathbf{c} = (-2, -1, -2) \Rightarrow d = \sqrt{4 + 1 + 4} = \sqrt{9} = 3.0000.$$

Ranking by distance (smaller = closer): $\mathbf{a}$ (1.41) < $\mathbf{b} = \mathbf{c}$ (3.00).

Compare the verdicts. On the very same vectors:

	cosine ($\uparrow$ better)	Euclidean ($\downarrow$ better)	verdict
a	0.8944	1.4142	both: closest
b	0.5270	3.0000	both: far
c (= 2q)	1.0000	3.0000	disagree

Cosine calls $\mathbf{c}$ a *perfect* match (identical direction); Euclidean calls it as far away as the unrelated $\mathbf{b}$. Both are “correct” — they answer different questions. The choice of metric is a modelling decision, not a detail.

Mini-example · same direction, far apart

$\mathbf{c} = 2\mathbf{q}$ points exactly along $\mathbf{q}$, yet $\|\mathbf{q} - \mathbf{c}\| = \|\mathbf{q} - 2\mathbf{q}\| = \|-\mathbf{q}\| = \|\mathbf{q}\| = 3$. The distance equals the query’s own length. Doubling a vector moves it a full $\|\mathbf{q}\|$ away from itself — which is why raw Euclidean distance is unsafe on un-normalised embeddings where length varies arbitrarily.

3 The identity that unifies L2 and cosine

Expand the squared distance of two *unit* vectors $\hat{\mathbf{u}}, \hat{\mathbf{v}}$ (each of norm 1):

$$\|\hat{\mathbf{u}} - \hat{\mathbf{v}}\|^2 = \hat{\mathbf{u}} \cdot \hat{\mathbf{u}} - 2\hat{\mathbf{u}} \cdot \hat{\mathbf{v}} + \hat{\mathbf{v}} \cdot \hat{\mathbf{v}} = 1 - 2\cos\theta + 1 = \boxed{2 - 2\cos\theta}.$$

So on normalised vectors, squared Euclidean distance is a strictly decreasing function of cosine: **larger cosine** $\Leftrightarrow$ **smaller distance**, always. The two metrics produce the *identical ranking*.

Mini-example · verifying $2 - 2\cos$ on document $\mathbf{a}$

Unit vectors: $\hat{\mathbf{q}} = (0.6667, 0.3333, 0.6667)$, $\hat{\mathbf{a}} = (0.4472, 0, 0.8944)$, with $\hat{\mathbf{q}} \cdot \hat{\mathbf{a}} = \cos = 0.8944$. Then

$$\|\hat{\mathbf{q}} - \hat{\mathbf{a}}\|^2 = 2 - 2(0.8944) = 0.2111, \quad \|\hat{\mathbf{q}} - \hat{\mathbf{a}}\| = \sqrt{0.2111} = 0.4595.$$

Direct computation of the unit-vector difference gives 0.4595 too. ✓ Once normalised, $\mathbf{c}$ collapses onto $\mathbf{q}$ ($2 - 2 \cdot 1 = 0$ distance) and the cosine/Euclidean disagreement disappears.

4 So which do you use?

- **Normalised embeddings:** cosine, dot product, and Euclidean all rank identically — pick whichever your vector DB optimises (usually dot product). The $2 - 2\cos$ identity is why an “L2 index” and a “cosine index” return the same neighbours on unit vectors.
- **Un-normalised embeddings where length is noise:** Euclidean (like dot product) is distorted by magnitude — normalise first, or you will rank by passage length.
- **Genuinely metric spaces** (coordinates, not directions — e.g. geo, some image features): Euclidean is the honest choice and cosine would throw away real information.

Equation cheat-sheet

$$d_2(\mathbf{u}, \mathbf{v}) = \sqrt{\sum_i (u_i - v_i)^2}, \quad d_2 \geq 0, \quad d_2 = 0 \iff \mathbf{u} = \mathbf{v}$$

$$\text{unit vectors: } \|\hat{\mathbf{u}} - \hat{\mathbf{v}}\|^2 = 2 - 2\cos\theta \Rightarrow \text{cosine and L2 give the same ranking}$$

Appendix A · Reference implementation

```
import numpy as np
```

```
q = np.array([2,1,2])
for name, v in [("a",[1,0,2]), ("b",[0,3,1]), ("c",[4,2,4])]:
    print(name, round(np.linalg.norm(q - np.array(v)), 4))
# a 1.4142  b 3.0  c 3.0  -> L2 ranks a closest; c (=2q) is FAR
```

Internal learning material. Numbers reproducible from the appendix code. v1.0